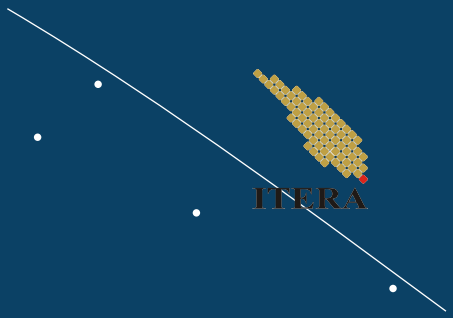
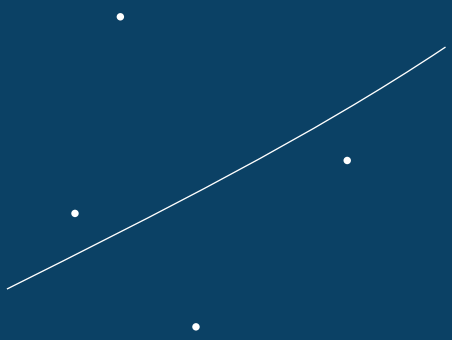


MODUL
PRAKTIKUM

MA25-21008



Program Studi Matematika
Fakultas Sains
Institut Teknologi Sumatera

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008

Semester Ganjil 2026/2027

Tim Pengajar

Rifky Fauzi
Triyana Muliawati, S.Si., M.Si.

Modul praktikum disusun untuk mendampingi materi kuliah, Lembar Kerja Mandiri, pre-test, dan persiapan UTS.

Contents

Identitas dan Cara Menggunakan Modul **iii**

1 Lingkungan Kerja Python **1**
- 1.1 Lingkungan kerja . . . 1
- 1.2 Struktur folder praktikum . . . 1
- 1.3 Pemeriksaan awal . . . 1

2 Praktikum 1: Python Dasar dan Alur Logika **3**
- 2.1 Tipe data dan operasi . . . 3
- 2.2 Percabangan sebagai representasi kondisi matematika . . . 3
- 2.3 Perulangan . . . 4
- 2.4 None, break, dan continue . . . 4
- 2.5 Loop bersarang . . . 4

3 Praktikum 2: Fungsi, Subprogram, dan Rekursi **6**
- 3.1 Subprogram dan fungsi . . . 6
- 3.2 Empat bentuk fungsi . . . 6
- 3.3 print, return, dan None . . . 6
- 3.4 Multiple return, tuple, dan unpacking . . . 7
- 3.5 Scope lokal dan global . . . 7
- 3.6 Komposisi dan fungsi sebagai objek . . . 7
- 3.7 Lambda . . . 8
- 3.8 Rekursi dasar . . . 8

4 Praktikum 3: Representasi Algoritma dan Struktur Data **10**
- 4.1 Pseudocode . . . 10
- 4.2 List, tuple, dan dictionary . . . 10
- 4.3 List comprehension . . . 10
- 4.4 NumPy array . . . 11
- 4.5 Loop dua dimensi dan matriks sebagai list . . . 11
- 4.6 Slicing dan beda hingga . . . 11

5 Praktikum 4: Debugging, Testing, Terminasi, dan Rekursi **13**
- 5.1 Debugging: kode benar secara sintaks belum tentu benar secara logika . . . 13
- 5.2 Exception handling . . . 13
- 5.3 Testing dengan assert . . . 13
- 5.4 Halt atau terminasi . . . 14
- 5.5 Rekursi dan call stack . . . 14
- 5.6 Iterasi Newton untuk akar kuadrat . . . 14

6 Praktikum 5: Kompleksitas Algoritma dan Pengukuran Kinerja **16**
- 6.1 Dari kode ke fungsi banyak kerja . . . 16
- 6.2 Contoh 1: jumlah $1 + 2 + \cdots + n$ . . . 16
  - 6.2.1 Versi loop . . . 16
  - 6.2.2 Menambah baris tetap . . . 17
  - 6.2.3 Versi rumus tertutup . . . 17
- 6.3 Contoh 2: kembali ke perhitungan statistik . . . 18

6.3.1 Rata-rata manual . . . 18
6.3.2 Statistik ringkas dan fungsi bawaan . . . 19
6.4 Contoh 3: struktur loop dan bentuk $T(n)$ . . . 19
6.4.1 Dua loop bersarang . . . 19
6.4.2 Dua loop berurutan . . . 19
6.4.3 Loop segitiga . . . 20
6.4.4 Loop yang membagi dua . . . 20
6.5 Mengukur waktu dengan `timeit` . . . 20
6.6 Contoh bersama: anagram . . . 21
6.6.1 Cara 1: sorting . . . 21
6.6.2 Cara 2: array frekuensi . . . 21
6.7 Pilihan latihan dari materi sebelumnya . . . 22
6.8 Latihan kelas: anagram dan `timeit` . . . 24

7 Praktikum 6: Searching, Sorting, dan Benchmark 26
7.1 Searching pada data terurut dan tidak terurut . . . 26
7.1.1 Tracing binary search . . . 26
7.2 Selection sort: arti satu *pass* . . . 27
7.2.1 Kode yang mencetak setiap proses pengulangan . . . 27
7.3 Bubble sort: setiap perbandingan dapat terlihat . . . 28
7.4 Merge sort: *divide, conquer, merge* . . . 29
7.4.1 Tahap 1: membagi masalah . . . 29
7.4.2 Tahap 2: menggabungkan dua list terurut . . . 29
7.4.3 Algoritma rekursif merge sort . . . 30
7.5 Sorting sebagai tahap sebelum binary search . . . 31
7.6 Benchmark sorting dengan `timeit` . . . 31

8 Praktikum 7: Aljabar Linier Komputasional 34
8.1 Aljabar linier dari sisi komputasi . . . 34
8.2 Vektor: indeks, operasi komponen, dan norma . . . 34
8.3 Operasi element-wise dan perkalian matriks . . . 35
8.4 Transpose sebagai pemindahan indeks . . . 37
8.5 Invers matriks 2×2: rumus, domain, dan verifikasi . . . 37
8.6 Sistem persamaan linier dan residual . . . 38
8.7 Nilai eigen: definisi, komputasi, dan verifikasi . . . 38
8.8 Principal Component Analysis sebagai komputasi matriks . . . 39

9 Latihan Terpadu Menjelang UTS 42

10 Praktikum 8: Regresi Linear Berbasis Aljabar Linier 44

11 Praktikum 9: Klasifikasi k-Nearest Neighbor 45

12 Praktikum 10: k-Means dan Mini Proyek Data 46

Daftar Pustaka Singkat 47

Identitas dan Cara Menggunakan Modul

Mata kuliah	Pemrograman untuk Ilmu Data dan Komputasi
Kode	MA25-21008
Program studi	Matematika
Fakultas	Fakultas Sains
Institusi	Institut Teknologi Sumatera
Perangkat lunak utama	Python 3, Jupyter Notebook/JupyterLab, Google Colab, atau IDE Python lain
Pustaka utama	Python standard library, NumPy, matplotlib, pandas pada materi setelah UTS
Dokumen pendamping	Materi pertemuan dan Lembar Kerja Mandiri (LKM)

Arah penggunaan

Modul ini bukan kumpulan sintaks. Setiap praktikum menghubungkan tiga hal: persoalan yang hendak diselesaikan, struktur algoritma, dan implementasi Python. Pada bagian sebelum UTS, latihan disusun mengikuti cakupan LKM: prediksi perilaku program, penelusuran alur, penulisan kode, debugging, analisis algoritma, dan komputasi matriks.

Untuk setiap praktikum, mahasiswa disarankan bekerja dengan urutan berikut.

1. Baca tujuan dan konsep inti sebelum membuka editor.
2. Kerjakan prediksi atau perhitungan manual untuk contoh kecil.
3. Tulis algoritma atau pseudocode jika masalah memiliki beberapa tahap.
4. Implementasikan program Python dengan nama variabel yang jelas.
5. Uji program pada beberapa masukan, termasuk kondisi batas.
6. Bandingkan hasil komputasi dengan hasil manual, identitas matematika, atau sifat algoritma yang diketahui.
7. Simpan kode dan hasil secara rapi agar dapat dijalankan ulang.

Peta praktikum

P	**Topik**	**Konsep utama**
1	Python dasar dan alur logika	tipe data, konversi, operasi, input/output, if, for, while, `None`, `break`, `continue`, loop bersarang

2	Fungsi, subprogram, dan rekursi	parameter, argumen, `return`, tuple/unpacking, scope, komposisi, fungsi sebagai objek, lambda, rekursi dasar
3	Representasi algoritma dan struktur data	pseudocode, list, tuple, dictionary, list comprehension, array NumPy, slicing, loop dua dimensi
4	Debugging, testing, dan terminasi	error logika, exception, `assert`, `print`/`return`/`None`, halt, rekursi, iterasi Newton
5	Kompleksitas algoritma dan pengukuran kinerja	fungsi kerja $T(n)$, Big-O, struktur loop, kodetimeit, anagram, hubungan teori dan waktu aktual
6	Searching, merge sort, dan benchmark	sequential/binary search, merge sort, tracing, invariant, `timeit`, benchmark yang adil
7	Aljabar linier komputasional	vektor, matriks, element-wise, matmul, transpose, invers, SPL, residual, norma, nilai eigen, data sebagai matriks
8	Regresi linear	matriks desain, least squares, residual, visualisasi
9	k-Nearest Neighbor	jarak, tetangga terdekat, klasifikasi sederhana, evaluasi
10	k-Means dan proyek data	centroid, iterasi, stopping rule, reproducibility, rancangan mini proyek

Bab 1

Lingkungan Kerja Python

Tujuan praktikum

Mahasiswa mampu menyiapkan lingkungan kerja Python, membuat struktur folder sederhana, menyimpan kode dan hasil, serta menjalankan ulang pekerjaan tanpa bergantung pada keadaan notebook sebelumnya.

1.1 Lingkungan kerja

Praktikum dapat dikerjakan menggunakan Jupyter Notebook, JupyterLab, Google Colab, Spyder, Visual Studio Code, atau IDE Python lain. Pilihan alat tidak mengubah logika program. Yang perlu dijaga adalah versi Python, pustaka yang digunakan, lokasi data, dan urutan eksekusi.

1.2 Struktur folder praktikum

Contoh sederhana:

```
praktikum_01/
  praktikum_01.ipynb
  data/
  output/
  README.md
```

Pada tahap awal, satu notebook sudah cukup. Pemisahan fungsi ke file lain dilakukan ketika materi fungsi dan subprogram telah dibahas.

1.3 Pemeriksaan awal

```
import sys
print(sys.version)
print(2 + 3)
print(type(2 + 3))
```

Latihan persiapan

1. Buat satu folder praktikum dengan struktur di atas.
2. Jalankan Python dan catat versinya.
3. Buat notebook yang menampilkan nama, NIM, dan hasil 2^{10}.

4. Ubah urutan eksekusi dua sel yang saling bergantung. Jelaskan mengapa notebook yang tidak dijalankan dari awal dapat memberi hasil yang membingungkan.
5. Restart kernel, lalu jalankan seluruh sel dari atas. Pastikan hasil tetap sama.
6. Buat `README.md` berisi cara menjalankan notebook dari awal.
7. Simpan satu keluaran teks ke folder `output/`.
8. Tuliskan satu aturan penamaan file yang akan Anda gunakan sepanjang semester.

Bab 2

Praktikum 1: Python Dasar dan Alur Logika

Tujuan praktikum

Mahasiswa mampu membaca dan menulis ekspresi Python, membedakan tipe data dasar, melakukan konversi tipe, menggunakan percabangan dan perulangan, serta menelusuri perubahan nilai variabel selama program berjalan.

2.1 Tipe data dan operasi

Empat tipe yang sering muncul pada awal perkuliahan adalah `int`, `float`, `bool`, dan `str`. Python menentukan perilaku operator berdasarkan tipe operand. Karena itu dua ekspresi yang tampak mirip dapat memiliki arti berbeda.

```
print(1 + True)       # bool berperilaku seperti 0/1 pada aritmetika
print('1' + '0')      # konkatenasi string
print(int('1') + 0)   # konversi eksplisit
print(3 / 2)          # pembagian biasa
print(3 // 2)         # floor division
print(bool('0'))      # string tidak kosong -> True
```

Prinsip membaca ekspresi

Sebelum menjalankan kode, tentukan tipe setiap operand, arti operator untuk tipe tersebut, lalu prediksi tipe dan nilai hasil. Jika operator tidak didefinisikan untuk pasangan tipe itu, prediksi error.

2.2 Percabangan sebagai representasi kondisi matematika

Untuk persamaan kuadrat $ax^2 + bx + c = 0$, diskriminan $D = b^2 - 4ac$ menentukan jenis akar. Struktur `if-elif-else` menerjemahkan kondisi matematika tersebut menjadi alur program.

```
a, b, c = 1.0, -3.0, 2.0
D = b**2 - 4*a*c

if D > 0:
    jenis = "dua akar real"
elif D == 0:
    jenis = "akar real kembar"
else:
```

```
    jenis = "akar kompleks"

print(D, jenis)
```

2.3 Perulangan

`for` cocok ketika himpunan nilai yang akan dilalui sudah jelas. `while` cocok ketika pengulangan bergantung pada kondisi yang berubah selama program berjalan.

```
data = [3, -2, 0, 7, -5, 4, 9]
total_positif = 0

for x in data:
    if x > 0:
        total_positif += x

print(total_positif)
```

2.4 None, break, dan continue

`None` dapat dipakai untuk menyatakan bahwa sebuah nilai tidak tersedia. Ini berbeda dari nol atau nilai ekstrem. `continue` melewati sisa langkah pada iterasi saat ini, sedangkan `break` menghentikan loop.

```
data = [2.5, None, -1.0, 4.2, None, 100.0, 3.1]
valid = []

for x in data:
    if x is None:
        continue
    if abs(x) > 50:
        break
    valid.append(x)

print(valid)
```

2.5 Loop bersarang

Loop bersarang diperlukan ketika proses memiliki dua indeks, misalnya membentuk tabel, pola karakter, atau matriks sederhana.

```
n = 5
for i in range(1, n + 1):
    for j in range(i):
        print('*', end='')
    print()
```

Jumlah karakter yang dicetak adalah $1 + 2 + \cdots + n = n(n+1)/2$.

Latihan Praktikum 1

1. Prediksi nilai dan tipe untuk `1 + True`, `True + True`, `'1' + '0'`, `'1' + 0`, `int('1') + 0`, `2**3.0`, dan `bool('0')`. Verifikasi setelah menulis prediksi.
2. Tulis program yang menerima bilangan bulat positif dan menampilkan bentuk desimal, biner, oktal, dan heksadesimal.
3. Untuk $x = 2.4$ dan $y = -1.2$, hitung $f(x, y) = \dfrac{e^{x-y} + \sqrt{x^2 + y^2}}{1 + |xy|}$ menggunakan modul `math`.
4. Buat program klasifikasi akar persamaan kuadrat berdasarkan diskriminan. Uji tiga kasus berbeda.
5. Implementasikan fungsi potongan secara langsung dengan `if-elif-else`: $x^2 + 1$ untuk $x < 0$, $2x + 1$ untuk $0 \le x < 3$, dan $\sqrt{x + 6}$ untuk $x \ge 3$.
6. Dari list `[3,-2,0,7,-5,4,9]`, hitung jumlah elemen positif, jumlah kuadrat elemen negatif, dan rata-rata elemen nonnol.
7. Gunakan `continue` untuk mengabaikan `None` dan hitung rata-rata data valid.
8. Gunakan `break` untuk menghentikan pemrosesan saat $|x| > 50$. Catat nilai akumulator pada setiap iterasi.
9. Buat segitiga bintang naik dan turun dengan loop bersarang.
10. Buat tabel perkalian 1 sampai n menggunakan dua loop.
11. Tulis program yang mencari faktor positif terkecil selain 1 dari bilangan $n > 1$ menggunakan `while`.
12. Gunakan faktor terkecil untuk menentukan apakah n prima. Uji 101, 221, dan 997.
13. Hitung $p_n = n \sin(\pi/n)$ untuk $n = 3, 4, 10, 100, 10^4, 10^6$ dan amati kecenderungannya.
14. Buat program yang membaca data sampai pengguna memasukkan kata `stop`. Hitung banyak data dan rata-rata.
15. Buat loop yang mencetak semua pasangan (i, j) dengan $1 \le i \le 4$, $1 \le j \le 4$, dan $i + j$ genap.
16. Jelaskan perbedaan makna data bernilai `0`, `None`, dan 100.0 dalam konteks pengukuran.
17. Tulis satu contoh loop `while` yang pasti berhenti dan satu yang tidak berhenti untuk input positif. Jelaskan alasannya.
18. Buat satu soal prediksi output sendiri yang menggunakan `if`, `for`, dan perubahan nilai variabel; tukarkan dengan teman untuk ditelusuri manual.

Bab 3

Praktikum 2: Fungsi, Subprogram, dan Rekursi

Tujuan praktikum

Mahasiswa mampu mendefinisikan fungsi dengan parameter dan nilai balik, memahami scope, menggunakan tuple/unpacking, menyusun fungsi dari fungsi lain, memperlakukan fungsi sebagai objek, serta mengenali struktur dasar rekursi.

3.1 Subprogram dan fungsi

Subprogram adalah bagian program yang diberi nama dan menjalankan tugas tertentu. Pada Python, fungsi yang dibuat dengan `def` merupakan bentuk subprogram yang paling umum. Fungsi membantu memisahkan persoalan besar menjadi unit yang dapat dipanggil, diuji, dan digunakan kembali.

```
def polinom_sederhana(x):
    return 2 - 3*x + x**2

print(polinom_sederhana(4))
```

Dalam definisi di atas, `x` adalah parameter. Pada pemanggilan `polinom_sederhana(4)`, nilai `4` adalah argumen.

3.2 Empat bentuk fungsi

Secara dasar fungsi dapat dibedakan berdasarkan ada tidaknya argumen dan ada tidaknya nilai balik.

Bentuk	Contoh	Makna
Tanpa argumen, tanpa return	`def salam(): print('Halo')`	melakukan aksi
Tanpa argumen, dengan return	`def pi(): return 3.14159`	menghasilkan nilai tetap
Dengan argumen, tanpa return	`def tampil(x): print(x)`	menerima input lalu menampilkan
Dengan argumen, dengan return	`def f(x): return x**2`	menghasilkan nilai untuk dipakai kembali

3.3 print, return, dan None

```python
def g(x):
    return x**2 - 1

def h(x):
    print(g(x))

y = h(3)
print(y)
```

`print` hanya menampilkan nilai. Jika fungsi tidak mencapai `return`, Python mengembalikan `None`. Karena itu hasil fungsi seperti `h` tidak langsung dapat dipakai dalam operasi aritmetika.

3.4 Multiple return, tuple, dan unpacking

```python
def stat_ringkas(a, b, c, d):
    nilai = [a, b, c, d]
    minimum = min(nilai)
    maksimum = max(nilai)
    rata = sum(nilai) / len(nilai)
    rentang = maksimum - minimum
    return minimum, maksimum, rata, rentang

mn, mx, r, rg = stat_ringkas(4, 7, 1, 9)
```

Python sebenarnya mengembalikan satu tuple. Unpacking menempatkan elemen tuple ke beberapa variabel.

3.5 Scope lokal dan global

```python
x = 10

def f(a):
    x = a + 2
    return x

print(f(5))
print(x)
```

Variabel `x` di dalam fungsi bersifat lokal. Untuk program komputasi, fungsi yang menerima input dan mengembalikan output secara eksplisit biasanya lebih mudah diuji daripada fungsi yang mengubah banyak variabel global.

3.6 Komposisi dan fungsi sebagai objek

Jika $(f \circ g)(x) = f(g(x))$, maka pola yang sama dapat ditulis sebagai fungsi yang memanggil fungsi lain.

```python
def proses(x, f):
    return f(x)

def kuadrat(x):
    return x**2

```

```
print(proses(5, kuadrat))
```

Nama `kuadrat` tanpa tanda kurung adalah objek fungsi dan dapat dikirim sebagai argumen.

3.7 Lambda

Lambda sesuai untuk fungsi pendek yang hanya terdiri dari satu ekspresi.

```
f = lambda x: 3*x**2 - 2*x + 1
print(f(2))
```

Jika logika mulai panjang, gunakan fungsi bernama dengan `def`.

3.8 Rekursi dasar

Rekursi terjadi ketika fungsi memanggil dirinya sendiri. Dua komponen wajib adalah *base case* dan langkah yang membawa masalah lebih dekat ke base case.

Untuk faktorial,

$$0! = 1, \qquad n! = n(n-1)!, \quad n \geq 1.$$

```
def faktorial(n):
    if n == 0:
        return 1
    return n * faktorial(n - 1)
```

Untuk $n = 4$, urutan pemanggilan adalah $4 \to 3 \to 2 \to 1 \to 0$, lalu nilai dikembalikan ke arah sebaliknya.

Latihan Praktikum 2

1. Buat masing-masing satu contoh untuk empat bentuk fungsi pada tabel di atas.
2. Tulis `polinom(x, koef)` untuk $p(x) = a_0 + a_1 x + \cdots + a_n x^n$ tanpa `numpy.polyval`.
3. Buat `stat_ringkas(a,b,c,d)` yang mengembalikan minimum, maksimum, rata-rata, dan rentang. Lakukan unpacking.
4. Ubah fungsi pada nomor sebelumnya agar menerima satu list data.
5. Prediksi keluaran program scope lokal dan global untuk beberapa nilai argumen.
6. Buat versi fungsi yang memakai `global`; jelaskan mengapa hasilnya dapat bergantung pada riwayat pemanggilan.
7. Tulis `jarak(p,q)` dan `energi(d)=1/(1+d^2)`, lalu buat `energi_dua_titik(p,q)` dengan komposisi.
8. Buat `proses(x,f)` lalu uji dengan fungsi kuadrat, kubik, nilai mutlak, dan fungsi linear.
9. Buat list fungsi `ops=[abs, kuadrat, tambah1]` dan terapkan berurutan pada $x = -3$.
10. Definisikan $f(x) = 3x^2 - 2x + 1$ dengan `lambda`, lalu evaluasi untuk $x = -3, -2, \ldots, 3$.
11. Bandingkan kode `map` dan list comprehension untuk evaluasi fungsi yang sama. Pilih yang lebih mudah dibaca dan jelaskan alasan Anda.
12. Diagnosa kode berikut: `def h(x): print(x**2)` kemudian `h(3)+1`. Mengapa gagal? Perbaiki.

13. Buat fungsi `ringkas_vektor(v)` yang mengembalikan $\|v\|_1$, $\|v\|_2$, $\max_i |v_i|$, dan rata-rata komponen.
14. Buat fungsi rekursif faktorial dan tulis jejak pemanggilan untuk $n = 5$.
15. Tulis fungsi iteratif faktorial. Bandingkan struktur kode dengan versi rekursif.
16. Untuk $a_0 = 1$ dan $a_n = 2a_{n-1} + 1$, tulis fungsi rekursif dan hitung a_0 sampai a_5.
17. Buat fungsi rekursif untuk menghitung jumlah $1 + 2 + \cdots + n$.
18. Buat satu fungsi rekursif yang salah karena tidak memiliki base case yang benar. Jelaskan mengapa program tidak berhenti secara wajar.
19. Tulis sebuah fungsi matematika pilihan Anda dengan input dan output yang jelas, tiga kasus uji, dan satu kondisi batas.
20. Ambil satu program panjang dari Praktikum 1 dan pecah menjadi paling sedikit dua fungsi yang mempunyai tanggung jawab berbeda.

Bab 4

Praktikum 3: Representasi Algoritma dan Struktur Data

Tujuan praktikum

Mahasiswa mampu menulis pseudocode, memilih struktur data sederhana, melakukan iterasi pada list/tuple/dictionary, menggunakan comprehension, dan membentuk serta memproses NumPy array satu dan dua dimensi.

4.1 Pseudocode

Pseudocode memisahkan logika algoritma dari sintaks Python. Bentuk yang digunakan di praktikum:

```
ALGORITMA NamaAlgoritma
INPUT:  data dan syarat masukan
OUTPUT: hasil
BEGIN
    inisialisasi
    IF kondisi THEN
        ...
    ENDIF
    FOR / WHILE ...
        ...
    ENDFOR / ENDWHILE
    RETURN hasil
END
```

4.2 List, tuple, dan dictionary

List cocok untuk koleksi berurutan yang dapat diubah. Tuple juga berurutan tetapi lazim dipakai untuk data yang dianggap tetap atau untuk multiple return. Dictionary menyimpan pasangan `key:value`.

```
nilai = {'A': 82, 'B': 71, 'C': 95, 'D': 64}
for nama, skor in nilai.items():
    print(nama, skor)
```

4.3 List comprehension

```
x = [-2, -1, 0, 1, 2]
y = [3*t**2 - 2*t + 1 for t in x]
```

Comprehension ringkas untuk transformasi sederhana. Untuk logika panjang atau bercabang, loop biasa sering lebih jelas.

4.4 NumPy array

Array NumPy bersifat homogen: elemen memiliki tipe data yang konsisten. Operasi aritmetika pada array dilakukan elemen per elemen secara alami.

```
import numpy as np
x = np.array([1, 4., -2, 7])
print(x.dtype)
print(2*x + 1)
```

4.5 Loop dua dimensi dan matriks sebagai list

```
A = []
for i in range(1, 5):
    baris = []
    for j in range(1, 5):
        baris.append(i + j + i*j)
    A.append(baris)
```

Setelah struktur dipahami, list of lists dapat dikonversi menjadi NumPy array untuk operasi numerik selanjutnya.

4.6 Slicing dan beda hingga

Jika y_i diukur pada waktu t_i, kecepatan rata-rata antar titik dapat ditulis

$$v_i = \frac{y_{i+1} - y_i}{t_{i+1} - t_i}.$$

Dengan NumPy, operasi ini dapat dilakukan menggunakan slicing.

```
v = (y[1:] - y[:-1]) / (t[1:] - t[:-1])
```

Latihan Praktikum 3

1. Tulis pseudocode untuk mencari nilai maksimum dan indeksnya dari sebuah list tanpa `max`.
2. Tulis pseudocode algoritma Euclid untuk $\gcd(a, b)$, lalu trace untuk $(252, 105)$.
3. Buat contoh iterasi list, tuple, dan dictionary. Jelaskan nilai yang dihasilkan pada setiap iterasi.
4. Dari dictionary nilai mahasiswa, hitung rata-rata dan key dengan nilai tertinggi.
5. Dengan list comprehension, hitung $y_i = 3x_i^2 - 2x_i + 1$ untuk $x = [-2, -1, 0, 1, 2]$.
6. Dari matriks list 3×3, ambil kolom terakhir dan diagonal utama tanpa NumPy.
7. Bentuk list berisi dua kali kuadrat elemen kolom tengah dengan comprehension.
8. Konversi list `[1,4.,-2,7]` menjadi NumPy array dan jelaskan `dtype` yang diperoleh.

9. Buat array integer dan float masing-masing panjang 5. Bandingkan hasil pembagian dengan 2.
10. Hitung $y_i = \sin(x_i) + x_i^2$ pada 1000 titik dengan loop Python dan NumPy. Pastikan hasil sama sampai toleransi tertentu.
11. Bentuk $A \in \mathbb{R}^{4\times4}$ dengan $a_{ij} = i + j + ij$ menggunakan dua loop.
12. Tentukan diagonal utama dan jumlah diagonal dari matriks nomor 11 tanpa NumPy.
13. Periksa secara manual dan dengan NumPy apakah matriks nomor 11 simetris.
14. Diberikan data yang mengandung `None`. Abaikan data hilang dan hentikan proses jika $|x| > 50$.
15. Bentuk dictionary skor-z dari dictionary nilai. Gunakan simpangan baku populasi dan verifikasi rata-rata skor-z mendekati nol.
16. Buat array waktu dan posisi sederhana, lalu hitung kecepatan rata-rata menggunakan slicing.
17. Jelaskan mengapa panjang array kecepatan satu lebih kecil dari panjang array posisi.
18. Buat matriks 5×5 dengan $a_{ij} = |i - j|$ menggunakan nested comprehension.
19. Bandingkan hasil loop biasa dan comprehension untuk transformasi data yang sama. Fokus pada keterbacaan, bukan hanya jumlah baris.
20. Tulis pseudocode dan kode Python untuk menghitung frekuensi kemunculan setiap kategori menggunakan dictionary.

Bab 5

Praktikum 4: Debugging, Testing, Terminasi, dan Rekursi

Tujuan praktikum

Mahasiswa mampu menemukan kesalahan logika, membedakan error tipe/scope/kondisi batas, menggunakan exception dan assert secara proporsional, menilai apakah loop atau rekursi berhenti, serta menelusuri algoritma iteratif dan rekursif.

5.1 Debugging: kode benar secara sintaks belum tentu benar secara logika

```
def rata(data):
    total = 0
    for x in data:
        total += x
        return total / len(data)
```

Kode ini valid secara sintaks, tetapi `return` berada terlalu awal. Uji dengan data kecil untuk menunjukkan bug sebelum memperbaikinya.

5.2 Exception handling

```
def rasio(a, b):
    try:
        x = float(a)
        y = float(b)
        return x / y
    except (TypeError, ValueError):
        return None
    except ZeroDivisionError:
        return None
```

Exception handling dipakai ketika kegagalan memang merupakan kondisi yang mungkin muncul dari input atau operasi. Jangan menangkap semua exception tanpa alasan karena bug pemrograman dapat tersembunyi.

5.3 Testing dengan assert

```python
def jarak_1d(x, y):
    return abs(x - y)

assert jarak_1d(3, 3) == 0
assert jarak_1d(3, 8) == 5
assert jarak_1d(-2, 3) == 5
```

Kasus uji yang baik mencakup kasus normal, kondisi batas, dan kasus yang hasilnya sudah diketahui.

5.4 Halt atau terminasi

Algoritma dikatakan berhenti untuk sebuah input jika eksekusinya selesai setelah sejumlah langkah berhingga. Untuk loop, salah satu cara memberi alasan terminasi adalah menemukan ukuran bilangan bulat nonnegatif yang berkurang menuju nol.

```python
i = n
while i > 0:
    i = i // 2
```

Untuk $n > 0$, nilai `i` mengecil hingga nol.

5.5 Rekursi dan call stack

Pada rekursi, setiap pemanggilan menunggu hasil pemanggilan yang lebih kecil. Base case menghentikan rantai pemanggilan.

5.6 Iterasi Newton untuk akar kuadrat

Untuk $f(x) = x^2 - n$, Newton-Raphson memberikan

$$x_{k+1} = x_k - \frac{x_k^2 - n}{2x_k} = \frac{1}{2}\left(x_k + \frac{n}{x_k}\right).$$

Contoh implementasi dengan jumlah iterasi tetap:

```python
def square_root(n):
    root = n / 2
    for k in range(20):
        root = 0.5 * (root + n / root)
    return root
```

Kriteria berhenti dapat diperbaiki dengan toleransi $|x_{k+1} - x_k| < \varepsilon$.

Latihan Praktikum 4

1. Temukan dan perbaiki bug pada fungsi rata-rata dengan `return` di dalam loop.
2. Buat dua test case yang pasti gagal pada versi salah dan lulus pada versi benar.
3. Diagnosa fungsi yang mencoba mengubah variabel global tanpa deklarasi `global`. Berikan perbaikan dengan dan tanpa variabel global.
4. Tambahkan pemeriksaan pada fungsi invers 2×2 agar matriks singular ditolak.
5. Tulis lima `assert` untuk fungsi jarak Euclid dua vektor.

6. Uji sifat simetri $d(x, y) = d(y, x)$ dan kasus $d((0, 0), (3, 4)) = 5$.
7. Tulis `rasio(a,b)` yang menangani `ValueError`, `TypeError`, dan `ZeroDivisionError`.
8. Jelaskan perbedaan `print(g(x))`, `return g(x)`, dan fungsi tanpa `return`.
9. Tentukan apakah loop `i=i//2`, `i=i-2`, dan `i=i+1` berhenti untuk semua input positif.
10. Untuk loop `i=i-2`, klasifikasikan input yang mencapai kondisi tertentu dan yang tidak.
11. Definisikan ukuran yang menurun untuk membuktikan terminasi loop pembagian dua.
12. Tulis fungsi rekursif untuk $a_0 = 2$, $a_n = 3a_{n-1} - 2$. Hitung a_1 sampai a_6.
13. Tulis fungsi rekursif untuk $u_0 = 1$, $u_1 = 3$, $u_n = u_{n-1} + 2u_{n-2}$. Gambar pohon pemanggilan untuk u_4.
14. Bandingkan faktorial rekursif dan iteratif untuk $n = 5$. Tulis call stack versi rekursif.
15. Jelaskan mengapa rekursi Fibonacci naif menghitung submasalah yang sama berkali-kali.
16. Gunakan iterasi Newton untuk menghitung $\sqrt{2}$, $\sqrt{9}$, dan $\sqrt{4563}$.
17. Modifikasi Newton agar berhenti berdasarkan toleransi, lalu kembalikan juga banyak iterasi.
18. Uji Newton untuk nilai awal berbeda. Apakah semua pilihan nilai awal layak?
19. Buat satu contoh fungsi yang gagal pada kondisi batas meskipun bekerja pada input biasa. Tulis test yang menangkap kegagalan tersebut.
20. Pilih satu fungsi dari praktikum sebelumnya dan tulis sekurangnya enam test case yang mencakup normal, batas, dan input tidak valid.

Bab 6

Praktikum 5: Kompleksitas Algoritma dan Pengukuran Kinerja

Tujuan praktikum

Mahasiswa mampu membaca program sebagai rangkaian kerja komputasi, membentuk fungsi banyak kerja $T(n)$ dari kode, menentukan orde pertumbuhan Big-O, membedakan $T(n)$ dari waktu eksekusi aktual, serta menguji prediksi teori melalui eksperimen Python.

6.1 Dari kode ke fungsi banyak kerja

Pada praktikum sebelumnya kita menulis program agar menghasilkan keluaran yang benar. Sekarang kita menambahkan satu pertanyaan: jika ukuran input diperbesar, berapa banyak pekerjaan yang dilakukan program?

Kita gunakan

$$T(n)$$

untuk menyatakan **fungsi banyak kerja terhadap ukuran input** n. Sebelum menghitung $T(n)$, tentukan terlebih dahulu operasi yang sedang dihitung: penjumlahan, assignment, perbandingan, akses elemen, atau eksekusi baris utama.

Catatan

$T(n)$ bukan waktu dalam detik. $T(n)$ diperoleh dari algoritma dan kode. Waktu aktual dapat diukur dengan `timeit`, tetapi dipengaruhi implementasi Python, komputer, proses lain pada sistem, dan bentuk input.

6.2 Contoh 1: jumlah $1 + 2 + \cdots + n$

6.2.1 Versi loop

```
def jumlah_loop(n):
    total = 0
    for i in range(1, n + 1):
        total = total + i
    return total
```

Dengan konvensi bahwa assignment, operasi aritmetika, dan return masing-masing dihitung sebagai satu kerja:

- `total = 0`: 1;
- `n + 1`: 1;
- `total = total + i`: 2 kerja dan terjadi n kali;
- `return total`: 1.

Maka

$$T(n) = 1 + 1 + 2n + 1 = 2n + 3,$$

sehingga

$$T(n) = O(n).$$

Untuk beberapa ukuran input:

n	1	5	10	100
$T(n) = 2n + 3$	5	13	23	203

6.2.2 Menambah baris tetap

```python
def jumlah_loop_dekoratif(n):
    print("Halo")
    print("Menghitung jumlah 1 sampai", n)
    print("----------------------------")

    total = 0
    for i in range(1, n + 1):
        total = total + i

    return total
```

Tiga `print` menambah pekerjaan tetap:

$$T(n) = 2n + 6.$$

Nilai $T(n)$ berubah, tetapi ordenya tetap

$$O(n).$$

6.2.3 Versi rumus tertutup

Identitas

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

memberikan algoritma lain.

```python
def jumlah_rumus(n):
    return n * (n + 1) // 2
```

Pada ekspresi tersebut terdapat satu penjumlahan, satu perkalian, satu pembagian bilangan bulat, dan satu return:

$$T(n) = 4, \qquad O(1).$$

Jadi persoalan matematika yang sama dapat diselesaikan dengan pertumbuhan kerja yang berbeda:

$$\boxed{\text{loop} : O(n)} \qquad \boxed{\text{rumus tertutup} : O(1)}.$$

6.3 Contoh 2: kembali ke perhitungan statistik

Pada Praktikum 2 kita telah membahas fungsi, parameter, `return`, dan statistik ringkas. Sekarang kita gunakan kembali program tersebut untuk melihat banyak kerjanya.

6.3.1 Rata-rata manual

```
def rata_manual(data):
    total = 0

    for x in data:
        total = total + x

    return total / len(data)
```

Misalkan n adalah banyak data. Dengan konvensi hitung yang sama:

- inisialisasi `total`: 1;
- penjumlahan dan assignment di dalam loop: $2n$;
- `len(data)`: 1;
- pembagian: 1;
- return: 1.

Maka

$$T(n) = 2n + 4, \qquad O(n).$$

Tambahkan counter untuk memeriksa berapa elemen yang benar-benar diproses.

```
def rata_dengan_counter(data):
    total = 0
    counter = 0

    for x in data:
        total = total + x
        counter = counter + 1

    return total / len(data), counter
```

Jalankan:

```
for n in [10, 100, 1000, 10000]:
    data = list(range(1, n + 1))
    rata, counter = rata_dengan_counter(data)
    print(n, rata, counter)
```

Sebelum menjalankan program, isi kolom teori:

n	$T(n)$	counter teori	counter program
10		10	
100		100	
1000		1000	
10000		10000	

6.3.2 Statistik ringkas dan fungsi bawaan

Perhatikan kembali fungsi:

```
def stat_ringkas(data):
    minimum = min(data)
    maksimum = max(data)
    rata = sum(data) / len(data)
    rentang = maksimum - minimum
    return minimum, maksimum, rata, rentang
```

Kode ini pendek, tetapi `min`, `max`, dan `sum` masing-masing tetap perlu menelusuri data. Untuk list berukuran n:

- `min(data)`: sekitar $n - 1$ perbandingan;
- `max(data)`: sekitar $n - 1$ perbandingan;
- `sum(data)`: sekitar n penjumlahan.

Jadi pekerjaan yang bergantung pada ukuran input tetap sebanding dengan

$$(n - 1) + (n - 1) + n,$$

dan

$$T(n) = O(n).$$

Satu baris Python belum tentu satu kerja

Fungsi bawaan dapat menyembunyikan loop. Analisis kompleksitas melihat pekerjaan yang dilakukan algoritma, bukan hanya banyak baris yang tertulis.

6.4 Contoh 3: struktur loop dan bentuk $T(n)$

6.4.1 Dua loop bersarang

```
def hitung_persegi(n):
    operasi = 0

    for i in range(n):
        for j in range(n):
            operasi += 1

    return operasi
```

Baris terdalam dijalankan

$$T(n) = n^2,$$

sehingga

$$O(n^2).$$

6.4.2 Dua loop berurutan

```
def hitung_berurutan(n):
    operasi = 0

    for i in range(n):
```

```
        operasi += 1

    for j in range(n):
        operasi += 1

    return operasi
```

Maka

$$T(n) = n + n = 2n, \qquad O(n).$$

6.4.3 Loop segitiga

```
def hitung_segitiga(n):
    operasi = 0

    for i in range(n):
        for j in range(i + 1):
            operasi += 1

    return operasi
```

Banyak eksekusi baris terdalam:

$$T(n) = 1 + 2 + \cdots + n = \frac{n(n+1)}{2},$$

sehingga

$$O(n^2).$$

6.4.4 Loop yang membagi dua

```
def hitung_bagi_dua(n):
    i = n
    operasi = 0

    while i > 0:
        operasi += 1
        i = i // 2

    return operasi
```

Untuk $n \geq 1$:

$$T(n) = \lfloor \log_2 n \rfloor + 1,$$

sehingga

$$O(\log n).$$

6.5 Mengukur waktu dengan `timeit`

Analisis $T(n)$ menjelaskan pertumbuhan kerja algoritma. Pengukuran waktu menjawab pertanyaan yang berbeda: berapa lama implementasi tertentu berjalan pada mesin tertentu?

```
from timeit import repeat
from statistics import median

def waktu_per_panggilan(f, arg, number=1000):
```

```
    hasil = repeat(
        lambda: f(arg),
        repeat=7,
        number=number
    )
    return median(hasil) / number
```

Contoh bersama:

```
for n in [10, 100, 1000, 10000]:
    t_loop = waktu_per_panggilan(jumlah_loop, n)
    t_rumus = waktu_per_panggilan(jumlah_rumus, n)
    print(n, t_loop, t_rumus)
```

Catatan

`timeit` tidak menghitung kompleksitas algoritma. Big-O diperoleh dari pertumbuhan $T(n)$. Data waktu hanya menunjukkan perilaku implementasi secara empiris. Jangan memasukkan `print` ke dalam fungsi yang sedang dibandingkan waktunya.

6.6 Contoh bersama: anagram

Dua string adalah anagram jika memiliki karakter dengan frekuensi yang sama, tetapi urutannya dapat berbeda.

6.6.1 Cara 1: sorting

```
def anagram_sort(a, b):
    if len(a) != len(b):
        return False

    return sorted(a) == sorted(b)
```

Untuk string panjang n, sorting memberi pertumbuhan

$$O(n \log n).$$

6.6.2 Cara 2: array frekuensi

Untuk contoh ini anggap input hanya berisi huruf kecil `a` sampai `z`.

```
def anagram_frekuensi(a, b):
    if len(a) != len(b):
        return False

    freq = [0] * 26

    for ch in a:
        indeks = ord(ch) - ord('a')
        freq[indeks] += 1

    for ch in b:
        indeks = ord(ch) - ord('a')
        freq[indeks] -= 1

```

```
    return all(x == 0 for x in freq)
```

Dua loop menelusuri masing-masing n karakter. Pemeriksaan terakhir memiliki 26 posisi:

$$T(n) = 2n + O(1), \qquad O(n).$$

Uji contoh kecil bersama:

```
pasangan = [
    ("listen", "silent"),
    ("data", "adat"),
    ("python", "typhon"),
    ("matematika", "matematika"),
]

for a, b in pasangan:
    print(a, b,
          anagram_sort(a, b),
          anagram_frekuensi(a, b))
```

Pengukuran waktu untuk string yang lebih panjang dikerjakan sebagai latihan kelas. Benchmark tidak dipakai sebagai pengganti analisis Big-O.

6.7 Pilihan latihan dari materi sebelumnya

Untuk setiap varian yang dipilih:

1. tentukan ukuran input n;
2. tentukan operasi utama yang dihitung;
3. bentuk $T(n)$;
4. tentukan Big-O;
5. jalankan algoritma untuk sedikitnya empat ukuran input;
6. sajikan tabel n, $T(n)$ teori, counter jika digunakan, dan waktu aktual;
7. jelaskan hubungan hasil program dengan analisis teori.

Varian A: statistik ringkas

Gunakan kembali fungsi `stat_ringkas`. Analisis pekerjaan di dalam `min`, `max`, dan `sum`. Jalankan untuk

$$n = 10,\ 100,\ 1000,\ 10000.$$

Varian B: statistik dalam satu lintasan

```
def stat_satu_pass(data):
    total = 0
    minimum = data[0]
    maksimum = data[0]

    for x in data:
        total += x
        if x < minimum:
```

```
            minimum = x
        if x > maksimum:
            maksimum = x

    rata = total / len(data)
    return minimum, maksimum, rata, maksimum - minimum
```

Bandingkan hasil dan banyak lintasan data dengan Varian A.

Varian C: jumlah data positif

```
def jumlah_positif(data):
    total = 0

    for x in data:
        if x > 0:
            total += x

    return total
```

Bedakan banyak iterasi, perbandingan, dan penjumlahan yang benar-benar terjadi.

Varian D: evaluasi polinom

```
def polinom(x, koef):
    hasil = 0

    for i in range(len(koef)):
        hasil = hasil + koef[i] * x**i

    return hasil
```

Ambil n sebagai banyak koefisien.

Varian E: maksimum tanpa `max`

```
def maksimum_manual(data):
    terbesar = data[0]

    for x in data[1:]:
        if x > terbesar:
            terbesar = x

    return terbesar
```

Hitung banyak perbandingan untuk beberapa ukuran n.

Varian F: membentuk matriks

```
def bentuk_matriks(n):
    A = []

    for i in range(n):
        baris = []

```

```
        for j in range(n):
            baris.append(i + j + i*j)

        A.append(baris)

    return A
```

Pilih operasi utama pada loop terdalam dan bentuk $T(n)$.

Varian G: mencari faktor terkecil

```
def faktor_terkecil(n):
    d = 2

    while d < n:
        if n % d == 0:
            return d
        d += 1

    return n
```

Bandingkan input prima dan komposit. Jelaskan mengapa banyak kerja dapat berbeda untuk nilai input yang berbeda.

Varian H: iterasi Newton

```
def akar_newton(n, tol=1e-10):
    x = n / 2
    iterasi = 0

    while True:
        x_baru = 0.5 * (x + n / x)
        iterasi += 1

        if abs(x_baru - x) < tol:
            return x_baru, iterasi

        x = x_baru
```

Catat banyak iterasi untuk beberapa nilai n dan toleransi. Jelaskan terlebih dahulu apa yang layak disebut ukuran input sebelum menyatakan kompleksitas.

6.8 Latihan kelas: anagram dan `timeit`

Buat pasangan anagram dengan panjang

$$n = 10^2,\ 10^3,\ 10^4,\ 10^5.$$

Gunakan input yang sama untuk kedua algoritma, lakukan pengukuran berulang, lalu sajikan:

n	waktu sorting	waktu frekuensi
10^2		
10^3		
10^4		
10^5		

Diskusikan apakah kecenderungan waktu masuk akal terhadap teori $O(n \log n)$ dan $O(n)$. Jelaskan mengapa satu pengukuran tidak cukup dan mengapa benchmark bukan definisi Big-O.

Latihan Praktikum 5

1. Urutkan $1, \log n, n, n \log n, n^2, n^3, 2^n, n!$ dari pertumbuhan paling lambat.
2. Hitung $T(n)$ untuk dua loop $n \times n$, dua loop berurutan, loop segitiga, dan loop pembagian dua.
3. Untuk `jumlah_loop` dan `jumlah_rumus`, hitung $T(n)$ dan tentukan Big-O. Verifikasi keluaran keduanya sama.
4. Tambahkan tiga operasi tetap di luar loop. Hitung kembali $T(n)$ dan jelaskan apa yang berubah.
5. Untuk `rata_manual`, isi tabel teori dan counter untuk $n = 10, 100, 1000, 10000$.
6. Pilih sekurangnya **dua** Varian A–H. Bentuk $T(n)$, jalankan sedikitnya empat ukuran input, dan sajikan hasil dalam tabel.
7. Untuk salah satu varian, tambahkan counter dan periksa apakah hasil program konsisten dengan $T(n)$.
8. Untuk salah satu varian, gunakan `timeit.repeat`. Bandingkan pola waktu dengan pola $T(n)$ tanpa menyamakan keduanya.
9. Uji kedua algoritma anagram pada sedikitnya lima pasangan string.
10. Lakukan benchmark anagram pada beberapa ukuran n sebagai latihan kelas dan buat tabel hasil.
11. Jelaskan *time-space trade-off* pada dua algoritma anagram.
12. Untuk
$$T_1(n) = 7n^2 + 3n + 40, \qquad T_2(n) = 0.001n^3 + 100n,$$
tentukan orde keduanya dan cari secara numerik rentang n ketika $T_1(n) < T_2(n)$.
13. Buat satu fungsi baru yang mengembangkan materi Praktikum 1–4. Tentukan ukuran input, operasi utama, $T(n)$, dan Big-O.
14. Tulis kesimpulan yang membedakan $T(n)$, Big-O, counter operasi, dan waktu aktual dari `timeit`.

Bab 7

Praktikum 6: Searching, Sorting, dan Benchmark

Tujuan praktikum

Mahasiswa mampu menelusuri proses searching dan sorting langkah demi langkah, membedakan arti satu *pass* pada beberapa algoritma sorting, mengimplementasikan selection sort, bubble sort, dan merge sort, menggunakan hasil sorting sebagai prasyarat binary search, serta menghubungkan pengukuran `timeit` dengan analisis kompleksitas algoritma.

7.1 Searching pada data terurut dan tidak terurut

Sequential search memeriksa elemen satu per satu sehingga dapat digunakan pada data yang belum terurut. Binary search membutuhkan data terurut karena pada setiap iterasi algoritma membuang separuh daerah pencarian.

```
def sequential_search(a, target):
    for i in range(len(a)):
        if a[i] == target:
            return i
    return -1


def binary_search(a, target):
    kiri, kanan = 0, len(a) - 1
    while kiri <= kanan:
        tengah = (kiri + kanan) // 2
        if a[tengah] == target:
            return tengah
        if a[tengah] < target:
            kiri = tengah + 1
        else:
            kanan = tengah - 1
    return -1
```

7.1.1 Tracing binary search

Untuk list terurut `[3,5,6,8,11,12,14,15,17,18]`, catat pada setiap iterasi: indeks kiri, indeks kanan, indeks tengah, nilai tengah, dan keputusan algoritma. Jika target lebih kecil dari nilai tengah, interval kanan dibuang; jika target lebih besar, interval kiri dibuang.

```
def binary_search_trace(a, target):
```

```
    kiri, kanan = 0, len(a) - 1
    langkah = 1

    while kiri <= kanan:
        tengah = (kiri + kanan) // 2
        print("langkah", langkah,
              "kiri=", kiri,
              "tengah=", tengah,
              "kanan=", kanan,
              "nilai=", a[tengah])

        if a[tengah] == target:
            print("target ditemukan")
            return tengah
        elif a[tengah] < target:
            kiri = tengah + 1
        else:
            kanan = tengah - 1

        langkah += 1

    print("target tidak ditemukan")
    return -1
```

7.2 Selection sort: arti satu *pass*

Pada selection sort menaik, satu *pass* berarti:

1. tentukan posisi pertama pada bagian yang belum terurut;
2. cari nilai minimum pada bagian tersebut;
3. tukar minimum dengan elemen pada posisi pertama bagian yang belum terurut;
4. setelah *pass* ke-k, sebanyak k elemen terkecil sudah berada pada posisi final di sebelah kiri.

Contoh awal:

$$[11, 7, 12, 14, 19, 1, 6, 18, 8, 20].$$

Tiga *pass* pertama adalah

pass	minimum yang dipilih	keadaan list setelah pass
1	1	$[1, 7, 12, 14, 19, 11, 6, 18, 8, 20]$
2	6	$[1, 6, 12, 14, 19, 11, 7, 18, 8, 20]$
3	7	$[1, 6, 7, 14, 19, 11, 12, 18, 8, 20]$

7.2.1 Kode yang mencetak setiap proses pengulangan

Kode berikut sengaja verbose untuk praktikum. Setiap perbandingan pada loop dalam dicetak sehingga mahasiswa dapat melihat bagaimana kandidat minimum berubah, lalu melihat hasil akhir satu *pass*.

```
def selection_sort_trace(data):
    a = data.copy()
    n = len(a)

    for i in range(n - 1):
        idx_min = i
```

```
        print(f"\nPASS {i+1}")
        print("awal :", a)

        for j in range(i + 1, n):
            print(f"bandingkan a[{j}]={a[j]} "
                  f"dengan minimum sementara "
                  f"a[{idx_min}]={a[idx_min]}")

            if a[j] < a[idx_min]:
                idx_min = j
                print("  -> minimum baru:", a[idx_min])

        a[i], a[idx_min] = a[idx_min], a[i]
        print("swap  :", a)
        print("final :", a[:i+1],
              "| belum terurut:", a[i+1:])

    return a
```

Catatan

Untuk memahami algoritma, gunakan versi `*_trace`. Untuk benchmark, gunakan versi tanpa `print`, karena operasi pencetakan jauh lebih mahal daripada perbandingan dan pertukaran yang sedang dianalisis.

7.3 Bubble sort: setiap perbandingan dapat terlihat

Pada bubble sort, satu *pass* adalah satu lintasan dari kiri ke kanan yang membandingkan pasangan bersebelahan. Jika urutannya salah, kedua elemen ditukar. Setelah satu *pass* penuh, elemen terbesar pada bagian aktif berada di posisi paling kanan bagian tersebut.

```
def bubble_sort_trace(data):
    a = data.copy()
    n = len(a)

    for p in range(n - 1):
        print(f"\nPASS {p+1}")

        for j in range(0, n - 1 - p):
            print(f"bandingkan {a[j]} dan {a[j+1]}")

            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
                print("  swap ->", a)
            else:
                print("  tetap ->", a)

        print("hasil pass:", a)

    return a
```

Perbedaan arti *pass*

Algoritma	Makna satu langkah utama
Selection sort	satu *pass*: mencari minimum pada bagian belum terurut, lalu satu pertukaran ke posisi final di kiri
Bubble sort	satu *pass*: satu lintasan pasangan bersebelahan; elemen terbesar bergerak ke kanan
Merge sort	lebih tepat ditelusuri sebagai level pembagian dan tahap penggabungan, bukan *pass* seperti selection/bubble

7.4 Merge sort: *divide, conquer, merge*

Merge sort tidak mencari minimum atau menukar pasangan bersebelahan. Algoritma bekerja dengan dua tahap besar.

7.4.1 Tahap 1: membagi masalah

Untuk list

$$[21, 1, 26, 45, 29, 28, 2, 9],$$

algoritma membelah list menjadi dua sampai setiap sublist berukuran satu:

$$\begin{aligned}&[21, 1, 26, 45, 29, 28, 2, 9]\\ &\quad\to [21, 1, 26, 45] \mid [29, 28, 2, 9]\\ &\quad\to [21, 1] \mid [26, 45] \mid [29, 28] \mid [2, 9]\\ &\quad\to [21]|[1]|[26]|[45]|[29]|[28]|[2]|[9].\end{aligned}$$

Sublist satu elemen sudah terurut secara trivial. Ini adalah *base case* rekursi.

7.4.2 Tahap 2: menggabungkan dua list terurut

Fungsi `merge(left,right)` selalu menerima dua list yang **sudah terurut**. Bandingkan elemen paling depan; pindahkan yang lebih kecil ke hasil; ulangi sampai salah satu list habis; lalu tambahkan sisa list lainnya.

Contoh:

$$[1, 21] + [26, 45] \longrightarrow [1, 21, 26, 45].$$

```
def merge(left, right, depth=0, trace=False):
    hasil = []
    i = j = 0
    indent = "  " * depth

    if trace:
        print(indent + "gabungkan:", left, "+", right)

    while i < len(left) and j < len(right):
        if trace:
            print(indent + "bandingkan", left[i], "dan", right[j])

        if left[i] <= right[j]:
            hasil.append(left[i])
```

```
            if trace:
                print(indent + "ambil kiri :", left[i])
            i += 1
        else:
            hasil.append(right[j])
            if trace:
                print(indent + "ambil kanan:", right[j])
            j += 1

        if trace:
            print(indent + "hasil sementara:", hasil)

    # Salah satu list sudah habis.
    # Sisa list lain ditambahkan SATU KALI setelah while selesai.
    if i < len(left):
        if trace:
            print(indent + "sisa kiri  :", left[i:])
        hasil.extend(left[i:])

    if j < len(right):
        if trace:
            print(indent + "sisa kanan :", right[j:])
        hasil.extend(right[j:])

    if trace:
        print(indent + "hasil merge :", hasil)

    return hasil
```

Catatan

Perhatikan indentasi dua baris `hasil.extend(...)`. Keduanya berada **di luar** loop `while`. Selama kedua list masih memiliki elemen, algoritma hanya memilih satu elemen terkecil pada setiap pengulangan. Sisa list baru ditambahkan setelah salah satu sisi habis. Jika `extend` diletakkan di dalam `while`, sisa elemen akan ditambahkan terlalu cepat dan hasil *merge* menjadi keliru.

7.4.3 Algoritma rekursif merge sort

1. Jika panjang list ≤ 1, kembalikan list tersebut.
2. Tentukan indeks tengah.
3. Urutkan bagian kiri secara rekursif.
4. Urutkan bagian kanan secara rekursif.
5. Gabungkan dua hasil terurut dengan `merge`.

```
def merge_sort(data, depth=0, trace=False):
    a = data.copy()
    indent = "  " * depth

    if len(a) <= 1:
        if trace:
            print(indent + "base :", a)
        return a

```

```
    mid = len(a) // 2

    if trace:
        print(indent + "split:", a,
              "->", a[:mid], "|", a[mid:])

    left = merge_sort(a[:mid], depth + 1, trace)
    right = merge_sort(a[mid:], depth + 1, trace)

    hasil = merge(left, right, depth=depth, trace=trace)

    if trace:
        print(indent + "selesai level ini:", hasil)

    return hasil


data = [21, 1, 26, 45, 29, 28, 2, 9]
urut = merge_sort(data, trace=True)
print("hasil akhir:", urut)
```

Mengapa $O(n \log n)$?

Pembagian menghasilkan sekitar $\log_2 n$ level karena ukuran list terus dibagi dua. Pada setiap level, total elemen yang diproses saat proses *merge* tetap sekitar n. Maka

$$T(n) \approx n \log_2 n, \qquad O(n \log n).$$

7.5 Sorting sebagai tahap sebelum binary search

Binary search hanya aman digunakan jika data sudah terurut. Karena itu, sebuah prosedur dapat memiliki dua tahap: **sorting terlebih dahulu**, kemudian **searching**. Untuk satu kali pencarian, biaya sorting dapat lebih besar daripada sequential search; tetapi jika data yang sama akan dicari berkali-kali, biaya sorting dapat menjadi investasi awal yang berguna.

Gunakan data berikut:

$$D = [11, 7, 12, 14, 19, 1, 6, 18, 8, 20].$$

Urutkan D dengan salah satu algoritma yang telah dibuat, kemudian gunakan binary search untuk mencari 14 dan 9. Catat proses sorting dan proses searching secara terpisah.

7.6 Benchmark sorting dengan `timeit`

Versi trace tidak digunakan untuk benchmark. Buat versi `selection_sort`, `bubble_sort`, dan `merge_sort` tanpa `print`. Gunakan data awal yang sama pada setiap algoritma.

```
import random
import timeit
from math import log2


def buat_data(n, seed=42):
    rng = random.Random(seed)
    return [rng.randint(0, 10**6) for _ in range(n)]

```

```
10
11 def ukur(sort_func, data):
12     # copy agar setiap algoritma menerima kondisi awal yang sama
13     hasil = timeit.repeat(
14         lambda: sort_func(data.copy()),
15         repeat=3,
16         number=1
17     )
18     return min(hasil)
```

Catatan

Jangan mencoba membuat list Python berukuran 10^{12} atau 10^{18}. Bahkan 10^8 integer dapat membutuhkan memori yang sangat besar. Ukuran ekstrem pada latihan digunakan untuk menilai apakah mahasiswa dapat membedakan **benchmark aktual** dan **prediksi berdasarkan model kompleksitas**.

Jika waktu aktual untuk $n_0 = 1000$ adalah t_0, gunakan model pertumbuhan untuk memperoleh estimasi kasar:

$$\hat{t}(n) = t_0 \left(\frac{n}{n_0}\right)^2$$

untuk selection/bubble sort, dan

$$\hat{t}(n) = t_0 \frac{n \log_2 n}{n_0 \log_2 n_0}$$

untuk merge sort.

```
1 def estimasi_n2(t0, n0, n):
2     return t0 * (n / n0)**2
3
4
5 def estimasi_nlogn(t0, n0, n):
6     return t0 * ((n * log2(n)) /
7                  (n0 * log2(n0)))
```

Latihan Praktikum 6

1. Jalankan `selection_sort_trace` pada `[11,7,12,14,19,1,6,18,8,20]`. Untuk tiga *pass* pertama, catat setiap kandidat minimum, posisi pertukaran, dan keadaan list setelah pertukaran.
2. Jalankan `bubble_sort_trace` pada `[11,7,12,14,19,1]`. Jelaskan perbedaan antara satu perbandingan, satu pertukaran, dan satu *pass* penuh.
3. Untuk `[21,1,26,45,29,28,2,9]`, tulis secara manual seluruh pembagian sampai *base case*, lalu lakukan penggabungan kembali sampai list terurut. Setelah itu cocokkan dengan keluaran `merge_sort(..., trace=True)`.
4. Gunakan data
$$D = [11, 7, 12, 14, 19, 1, 6, 18, 8, 20].$$
Urutkan D dengan selection sort dan merge sort. Pastikan kedua hasil sama. Gunakan hasil yang sudah terurut untuk binary search terhadap target 14 dan 9. Jelaskan mengapa binary search tidak boleh langsung diterapkan pada D sebelum sorting.
5. Diberikan dictionary berikut.

```
1 mahasiswa = {
```

```
    "Nadia Prameswari": "125160047",
    "Raka Mahendra":    "125160012",
    "Salsa Nirmala":    "125160083",
    "Dimas Ardiansyah": "125160025",
    "Keisha Ramadhani": "125160061",
    "Farel Wicaksana":  "125160008",
    "Tania Maharani":   "125160074",
    "Bagas Pratama":    "125160039"
}
```

Rancang fungsi `urutkan_berdasarkan_nim(data)` yang mengembalikan **dictionary baru** dengan urutan entri berdasarkan NIM dari kecil ke besar. Untuk versi utama, jangan gunakan `sorted`; gunakan salah satu algoritma sorting yang dipelajari terhadap pasangan `(nama, nim)`. Jelaskan mengapa dictionary hasil tetap dapat mempertahankan urutan penyisipan pada Python modern.

6. Dengan dictionary yang sama, rancang `urutkan_berdasarkan_nama(data)` yang mengembalikan dictionary baru dengan nama terurut alfabetis. Bandingkan bagian algoritma yang berubah dengan pengurutan berdasarkan NIM.

7. Setelah dictionary diurutkan berdasarkan NIM, buat fungsi pencarian NIM tertentu yang mengembalikan nama mahasiswa. Pilih apakah sequential search atau binary search lebih tepat terhadap representasi data yang Anda buat, lalu jelaskan prasyaratnya.

8. Buat bilangan bulat acak dan ukur waktu sorting dengan `timeit`. Gunakan `n=1000` untuk benchmark aktual selection sort, bubble sort, dan merge sort. Gunakan pola berikut sebagai petunjuk:

```
data = buat_data(1000, seed=42)
t = ukur(selection_sort, data)
print(t)
```

Ulangi untuk fungsi sorting lainnya dengan **data awal yang sama**.

9. Pertimbangkan ukuran

$$n = 10^3, \quad 10^8, \quad 10^{12}, \quad 10^{18}.$$

Untuk 10^3, isi waktu aktual dari `timeit`. Untuk ukuran yang tidak feasible dibuat di memori atau dijalankan sampai selesai, tulis *tidak feasible* pada kolom waktu aktual dan hitung **estimasi waktu** dari model $O(n^2)$ atau $O(n \log n)$ menggunakan fungsi `estimasi_n2` dan `estimasi_nlogn`.

10. Sajikan hasil nomor 8–9 dalam tabel berikut dan interpretasikan perbedaannya.

n	selection	bubble	merge	keterangan
10^3				
10^8				
10^{12}				
10^{18}				

Bedakan dengan jelas nilai hasil `timeit` dan nilai hasil estimasi matematis.

11. Untuk masing-masing algoritma sorting, tentukan operasi utama, bentuk $T(n)$, dan Big-O. Hubungkan hasil teori tersebut dengan kecenderungan estimasi waktu pada nomor 9.

12. Jelaskan mengapa `print` tidak boleh berada di dalam fungsi yang sedang diukur dengan `timeit`, walaupun `print` justru sangat berguna pada versi trace untuk memahami proses algoritma.

Bab 8

Praktikum 7: Aljabar Linier Komputasional

Tujuan praktikum

Setelah praktikum ini, mahasiswa mampu menerjemahkan operasi aljabar linier menjadi struktur data dan algoritma, menelusuri indeks yang digunakan, menganalisis fungsi kerja sederhana, serta memverifikasi hasil komputasi menggunakan residual dan norma. Topik meliputi vektor, perkalian matriks, transpose, invers, sistem persamaan linier, nilai eigen, dan PCA.

8.1 Aljabar linier dari sisi komputasi

Pada mata kuliah aljabar linier, objek yang dipelajari antara lain vektor, matriks, transformasi linier, sistem persamaan linier, dan pasangan eigen. Dalam pemrograman, pertanyaan yang muncul sedikit berbeda:

- bagaimana objek matematika tersebut disimpan sebagai list atau array;
- indeks mana yang dibaca pada setiap operasi;
- berapa loop yang diperlukan dan bagaimana banyak kerjanya bertambah;
- bagaimana memastikan hasil numerik konsisten dengan persamaan matematisnya.

Prinsip kerja

Rumus aljabar linier tidak diganti oleh program. Program menerjemahkan rumus menjadi **struktur data, indeks, algoritma, dan pemeriksaan hasil**.

8.2 Vektor: indeks, operasi komponen, dan norma

Misalkan

$$x = (x_1, \ldots, x_n), \qquad y = (y_1, \ldots, y_n).$$

Tiga operasi dasar yang akan dipakai berulang adalah

$$x + y = (x_i + y_i)_{i=1}^{n}, \qquad x \cdot y = \sum_{i=1}^{n} x_i y_i,$$

dan jarak Euclidean

$$\|x - y\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}.$$

Kode berikut sengaja ditulis tanpa `np.dot` atau `np.linalg.norm` agar hubungan antara rumus dan indeks terlihat.

```
def dot(x, y):
    if len(x) != len(y):
        raise ValueError("dimensi berbeda")

    total = 0.0
    for i in range(len(x)):
        total += x[i] * y[i]
    return total


def norm_diff(x, y):
    if len(x) != len(y):
        raise ValueError("dimensi berbeda")

    total = 0.0
    for i in range(len(x)):
        d = x[i] - y[i]
        total += d * d
    return total ** 0.5
```

Jika panjang vektor adalah n, loop membaca setiap komponen satu kali. Karena itu

$$T(n) = c_0 + c_1 n \qquad \Longrightarrow \qquad T(n) \in O(n).$$

Setelah prinsipnya dipahami, hasil dapat dibandingkan dengan NumPy.

```
import numpy as np

x = np.array([1., -2., 3.])
y = np.array([4.,  0., -1.])

print(x + y)
print(np.dot(x, y))
print(np.linalg.norm(x - y))
```

8.3 Operasi element-wise dan perkalian matriks

Untuk

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 2 & 0 \\ 1 & 5 \end{pmatrix},$$

perkalian element-wise membaca posisi yang sama,

$$A \odot B = \begin{pmatrix} 1 \cdot 2 & 2 \cdot 0 \\ 3 \cdot 1 & 4 \cdot 5 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 3 & 20 \end{pmatrix},$$

sedangkan perkalian matriks membaca **baris pada A dan kolom pada B**,

$$AB = \begin{pmatrix} 1 \cdot 2 + 2 \cdot 1 & 1 \cdot 0 + 2 \cdot 5 \\ 3 \cdot 2 + 4 \cdot 1 & 3 \cdot 0 + 4 \cdot 5 \end{pmatrix} = \begin{pmatrix} 4 & 10 \\ 10 & 20 \end{pmatrix}.$$

Dalam NumPy, kedua operasi tersebut berbeda.

```
A * B   # element-wise
```

```
A @ B   # matrix product
```

Algoritma perkalian matriks

Jika $A \in \mathbb{R}^{m\times p}$ dan $B \in \mathbb{R}^{p\times n}$, maka $C = AB \in \mathbb{R}^{m\times n}$ dengan

$$c_{ij} = \sum_{k=1}^{p} a_{ik} b_{kj}.$$

Pseudocode-nya adalah:

```
INPUT : A berukuran m x p, B berukuran p x n
OUTPUT: C = AB berukuran m x n

cek banyak kolom A = banyak baris B
buat C berisi nol, ukuran m x n
FOR i = 0 sampai m-1:
    FOR j = 0 sampai n-1:
        s = 0
        FOR k = 0 sampai p-1:
            s = s + A[i][k] * B[k][j]
        C[i][j] = s
RETURN C
```

Implementasi langsung dari pseudocode:

```python
def matmul_manual(A, B):
    m = len(A)
    p = len(A[0])
    p2 = len(B)
    n = len(B[0])

    if p != p2:
        raise ValueError("dimensi tidak cocok")

    C = [[0.0 for j in range(n)] for i in range(m)]

    for i in range(m):
        for j in range(n):
            s = 0.0
            for k in range(p):
                s += A[i][k] * B[k][j]
            C[i][j] = s

    return C
```

Fungsi kerja

Dari struktur kode di atas, kerja dominan berada pada loop terdalam. Salah satu model hitungan kerja sederhana adalah

$$W(m,p,n) = 5 + 3mn + 2mnp.$$

Untuk matriks bujur sangkar $n \times n$,

$$T(n) = 5 + 3n^2 + 2n^3 \qquad \Longrightarrow \qquad T(n) \in O(n^3).$$

Perhatikan bahwa orde tersebut diperoleh dari **struktur kode**, bukan dari nama operasinya.

8.4 Transpose sebagai pemindahan indeks

Untuk $A \in \mathbb{R}^{m\times n}$, transpose $A^T \in \mathbb{R}^{n\times m}$ memenuhi

$$(A^T)_{ji} = A_{ij}.$$

Artinya, algoritma transpose cukup memindahkan setiap elemen dari posisi (i, j) ke (j, i).

```
def transpose(A):
    m = len(A)
    n = len(A[0])

    T = [[0 for i in range(m)] for j in range(n)]
    for i in range(m):
        for j in range(n):
            T[j][i] = A[i][j]

    return T
```

Ada mn elemen yang dipindahkan, sehingga biaya komputasinya sebanding dengan jumlah elemen:

$$T(m, n) \in O(mn).$$

Verifikasi dengan `np.array(A).T` setelah implementasi manual bekerja.

8.5 Invers matriks 2×2: rumus, domain, dan verifikasi

Untuk

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix},$$

invers hanya ada jika

$$\det(A) = ad - bc \neq 0,$$

dan

$$A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

Program harus memeriksa syarat tersebut sebelum menghitung invers.

```
def inv2(A):
    a, b = A[0]
    c, d = A[1]

    det = a*d - b*c
    if det == 0:
        raise ValueError("matriks singular")

    return [[ d/det, -b/det],
            [-c/det,  a/det]]
```

Hasil numerik tidak cukup diperiksa dari bentuk matriksnya. Verifikasi identitas

$$AA^{-1} \approx I.$$

Pada komputer gunakan toleransi, bukan kesamaan desimal mentah.

```
A = np.array([[2., 1.], [1., 1.]])
Ainv = np.array(inv2(A.tolist()))

print(A @ Ainv)
print(np.allclose(A @ Ainv, np.eye(2)))
```

8.6 Sistem persamaan linier dan residual

Untuk sistem

$$Ax = b,$$

solusi numerik $\hat{x}$ harus diperiksa melalui residual

$$r = b - A\hat{x},$$

kemudian ukur besar residual dengan norma

$$\|r\|_2.$$

Jika $\|r\|_2$ kecil, maka $A\hat{x}$ dekat dengan b dan solusi konsisten dengan sistem yang diberikan.

```
A = np.array([[2., 1.], [5., 3.]])
b = np.array([1., 4.])

x = np.linalg.solve(A, b)
r = b - A @ x
err = np.linalg.norm(r)

print("x =", x)
print("residual =", r)
print("norma residual =", err)
```

Catatan

Untuk menyelesaikan $Ax = b$, gunakan `np.linalg.solve(A,b)`. Jangan membentuk A^{-1} hanya untuk menghitung $A^{-1}b$. Pada matriks besar, pembentukan invers eksplisit menambah pekerjaan dan dapat memperburuk kestabilan numerik.

Sebagai gambaran biaya, invers matriks $n \times n$ dapat dipandang sebagai penyelesaian

$$AX = I,$$

yaitu banyak sistem linier. Algoritma langsung berbasis eliminasi Gaussian atau faktorisasi memiliki orde utama kubik,

$$T(n) \in O(n^3),$$

sedangkan penyimpanan matriks padat memerlukan orde $O(n^2)$ memori.

8.7 Nilai eigen: definisi, komputasi, dan verifikasi

Pasangan eigen (λ, v) memenuhi

$$Av = \lambda v, \qquad v \neq 0.$$

Untuk matriks kecil, nilai eigen dapat diperoleh dari persamaan karakteristik

$$\det(A - \lambda I) = 0.$$

Sebagai contoh,

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

memberikan

$$(2 - \lambda)^2 - 1 = 0, \qquad \lambda = 1, 3.$$

Komputasi dan verifikasi numerik:

```
A = np.array([[2., 1.], [1., 2.]])
w, V = np.linalg.eig(A)

for k in range(len(w)):
    lam = w[k]
    v = V[:, k]
    cek = np.linalg.norm(A @ v - lam * v)
    print(lam, v, cek)
```

Jika $\|Av - \lambda v\|_2$ kecil, pasangan (λ, v) konsisten secara numerik.

8.8 Principal Component Analysis sebagai komputasi matriks

Misalkan satu dataset memiliki m observasi dan p fitur,

$$X \in \mathbb{R}^{m \times p}.$$

Ketika p besar, visualisasi dan komputasi menjadi lebih sulit. Selain itu, beberapa fitur dapat membawa informasi yang hampir sama. PCA membentuk fitur baru (*principal components*) sebagai kombinasi linier fitur lama sehingga data dapat diringkas dari p fitur menjadi k komponen, dengan $k < p$.

Langkah matematis PCA

1. **Pusatkan data.** Untuk setiap fitur j,

$$\bar{x}_j = \frac{1}{m} \sum_{i=1}^{m} X_{ij}, \qquad X_c = X - \mathbf{1}\bar{x}^T.$$

2. **Hitung matriks kovariansi.**

$$S = \frac{1}{m-1} X_c^T X_c.$$

3. **Cari eigenvalue dan eigenvector.**

$$Sv_i = \lambda_i v_i.$$

4. **Ambil k arah utama.** Susun eigenvector dengan eigenvalue terbesar menjadi

$$W = (v_1 \; v_2 \; \cdots \; v_k) \in \mathbb{R}^{p \times k}.$$

5. **Proyeksikan data.**

$$Z = X_c W, \qquad Z \in \mathbb{R}^{m \times k}.$$

Dengan demikian, hubungan antara materi sebelumnya terlihat langsung: PCA menggunakan rata-rata kolom, transpose, perkalian matriks, matriks simetris, dan pasangan eigen.

Implementasi langkah dasar

```
X = np.array([
    [1., 2., 0.],
    [2., 1., 1.],
    [4., 3., 2.],
    [3., 4., 1.]
])

# 1. centering
xbar = X.mean(axis=0)
Xc = X - xbar

# 2. covariance matrix
S = (Xc.T @ Xc) / (len(X) - 1)
print("S simetris =", np.allclose(S, S.T))

# 3. eigen
w, V = np.linalg.eigh(S)

# urutkan dari eigenvalue terbesar
idx = np.argsort(w)[::-1]
w = w[idx]
V = V[:, idx]

# 4-5. ambil dua komponen dan proyeksikan
W = V[:, :2]
Z = Xc @ W

print("eigenvalue =", w)
print("Z =\n", Z)
```

Untuk X_c berukuran $m \times p$, pembentukan $X_c^T X_c$ mengikuti pola perkalian matriks $p \times m$ dengan $m \times p$. Kerja dominannya berorde

$$O(mp^2).$$

Latihan Praktikum 7

Kerjakan berurutan. Untuk setiap bagian, tampilkan hasil numerik dan penjelasan singkat mengenai operasi matematika yang sedang diterjemahkan ke dalam kode.

1. **Vektor dan norma.** Untuk $x = (1, -2, 3)$ dan $y = (4, 0, -1)$:
   (a) hitung manual $x + y$, $x \cdot y$, dan $\|x - y\|_2$;
   (b) verifikasi menggunakan fungsi `dot` dan `norm_diff`;
   (c) tuliskan $T(n)$ dan Big-O kedua fungsi tersebut.
2. **Perkalian matriks.** Gunakan

$$A = \begin{pmatrix} 1 & 2 & 0 \\ 3 & 1 & 2 \end{pmatrix}, \qquad B = \begin{pmatrix} 2 & 1 \\ 0 & 3 \\ 1 & 4 \end{pmatrix}.$$

   (a) tentukan ukuran AB sebelum menghitung;

(b) hitung satu elemen, misalnya c_{12}, secara manual;

(c) jalankan `matmul_manual(A,B)` dan bandingkan dengan `np.array(A) @ np.array(B)`;

(d) jelaskan dari tiga loop mengapa untuk matriks $n \times n$ kompleksitasnya $O(n^3)$.

3. **Transpose.** Implementasikan transpose dengan loop, uji pada matriks 2×3, lalu verifikasi menggunakan `.T`. Jelaskan mengapa biayanya $O(mn)$.

4. **Invers dan SPL.** Gunakan

$$A = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 4 \end{pmatrix}.$$

(a) hitung determinan dan invers 2×2 secara manual;

(b) verifikasi $AA^{-1} \approx I$ dengan `np.allclose`;

(c) selesaikan $Ax = b$ dengan `np.linalg.solve`;

(d) hitung $r = b - A\hat{x}$ dan $\|r\|_2$;

(e) ganggu satu komponen $\hat{x}$ sebesar 10^{-3} dan bandingkan norma residualnya.

5. **Nilai eigen.** Untuk

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix},$$

hitung nilai eigen dari $\det(A - \lambda I) = 0$, kemudian gunakan `np.linalg.eig` dan verifikasi setiap pasangan eigen menggunakan $\|Av - \lambda v\|_2$.

6. **PCA langkah demi langkah.** Gunakan data

$$X = \begin{pmatrix} 1 & 2 & 0 \\ 2 & 1 & 1 \\ 4 & 3 & 2 \\ 3 & 4 & 1 \end{pmatrix}.$$

Kerjakan berurutan:

(a) hitung rata-rata setiap kolom dan bentuk X_c;

(b) hitung $S = \frac{1}{m-1} X_c^T X_c$;

(c) buktikan secara numerik bahwa S simetris;

(d) hitung eigenvalue dan eigenvector S;

(e) urutkan eigenvector dari eigenvalue terbesar;

(f) bentuk W dari dua eigenvector utama dan hitung $Z = X_c W$;

(g) tuliskan dimensi X, X_c, S, W, dan Z.

7. **Latihan terpadu.** Diberikan

$$A = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 4 \\ 2 \end{pmatrix}.$$

Tulis pseudocode perkalian matriks-vektor Ax, implementasikan tanpa NumPy, tentukan fungsi kerjanya untuk $A \in \mathbb{R}^{n \times n}$, selesaikan sistem dengan `np.linalg.solve`, lalu hitung residual dan norma residual.

Bab 9

Latihan Terpadu Menjelang UTS

Bagian ini digunakan setelah Praktikum 1-7. Soal sengaja mencampur prediksi, tracing, debugging, penulisan program, analisis algoritma, dan komputasi matriks.

Latihan terpadu

1. Prediksi nilai dan tipe beberapa ekspresi yang mencampur `int`, `float`, `bool`, dan `str`. Sertakan satu contoh yang menghasilkan error.
2. Tulis program yang menerima koefisien persamaan kuadrat dan mengklasifikasikan jenis akar.
3. Trace loop yang menggunakan `continue` untuk `None` dan `break` untuk nilai ekstrem.
4. Tulis nested loop yang menghasilkan pola segitiga dan hitung jumlah total iterasi dalam bentuk rumus.
5. Buat fungsi yang mengembalikan empat ringkasan statistik dan lakukan unpacking.
6. Diagnosa fungsi yang hanya melakukan `print` tetapi hasilnya dipakai dalam ekspresi aritmetika.
7. Trace scope lokal dan global pada fungsi dengan nama variabel yang sama.
8. Gunakan fungsi sebagai argumen untuk menerapkan tiga transformasi pada satu nilai.
9. Tulis fungsi rekursif dari suatu relasi rekurensi dan tunjukkan base case.
10. Tulis pseudocode Euclid dan trace untuk satu pasangan bilangan.
11. Bentuk matriks sederhana dengan nested loop dan konversi ke NumPy.
12. Gunakan slicing untuk menghitung beda hingga dari dua array.
13. Temukan bug pada fungsi rata-rata yang memiliki `return` di lokasi salah.
14. Tulis `assert` untuk fungsi jarak yang mencakup kondisi batas.
15. Tentukan apakah tiga loop berbeda berhenti untuk semua input positif dan berikan alasan.
16. Turunkan iterasi Newton untuk $\sqrt{n}$ dari Newton-Raphson umum, lalu tulis implementasinya.
17. Tentukan Big-O dua loop bersarang, dua loop berurutan, dan loop yang membagi ukuran dengan dua.
18. Untuk dua model biaya polinomial, tentukan orde dan cari titik silang numerik.
19. Trace tiga pass selection sort pada list yang diberikan.
20. Trace binary search untuk key yang ditemukan dan key yang tidak ditemukan.

21. Gambar dua atau tiga level awal merge sort untuk list panjang 16.
22. Jelaskan perbedaan analisis kompleksitas dan benchmark empiris.
23. Hitung operasi vektor, norma, dan dot product secara manual lalu verifikasi dengan NumPy.
24. Bandingkan `A*B` dan `A@B` pada dua matriks 2×2.
25. Selesaikan SPL dan hitung norma residual.
26. Hitung nilai eigen matriks simetris 2×2 dan verifikasi pasangan eigen numerik.
27. Centering sebuah matriks data dan hitung $X_c^T X_c$; jelaskan mengapa hasilnya simetris.

Bab 10

Praktikum 8: Regresi Linear Berbasis Aljabar Linier

Tujuan praktikum

Mahasiswa mampu membentuk matriks desain, menghitung parameter least squares sederhana, membuat prediksi, menghitung residual, dan memvisualisasikan hasil regresi.

Untuk model $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$, bentuk matriks

$$y = X\beta + \varepsilon,$$

dan cari β melalui persamaan normal $(X^T X)\beta = X^T y$ untuk tujuan pembelajaran aljabar linier.

```
import numpy as np

x = np.array([1., 2., 3., 4., 5.])
y = np.array([2.1, 3.9, 6.2, 8.0, 10.1])
X = np.column_stack([np.ones(len(x)), x])
beta = np.linalg.solve(X.T @ X, X.T @ y)
yhat = X @ beta
residual = y - yhat
print(beta)
print(np.linalg.norm(residual))
```

Latihan Praktikum 8

1. Bentuk matriks desain untuk regresi dengan intercept.
2. Hitung $X^T X$ dan $X^T y$ secara manual untuk dataset kecil.
3. Selesaikan parameter dengan `np.linalg.solve`.
4. Hitung prediksi dan residual.
5. Hitung SSE, RMSE, dan R^2.
6. Buat scatter plot data dan garis regresi.
7. Bandingkan hasil dengan `numpy.polyfit`.
8. Ubah skala x dan amati perubahan parameter.
9. Tambahkan satu outlier dan amati perubahan model.
10. Jelaskan hubungan residual dengan konsep jarak dari data ke model.

Bab 11

Praktikum 9: Klasifikasi k-Nearest Neighbor

Tujuan praktikum

Mahasiswa mampu menggunakan jarak Euclid untuk mencari tetangga terdekat, melakukan voting kelas sederhana, dan mengevaluasi prediksi pada dataset kecil.

```python
import numpy as np

def jarak(x, y):
    return np.linalg.norm(x - y)
```

Latihan Praktikum 9

1. Hitung jarak sebuah titik uji ke semua data latih.
2. Urutkan indeks berdasarkan jarak.
3. Implementasikan k-NN untuk $k = 1$ dan $k = 3$.
4. Uji pengaruh perubahan skala salah satu fitur.
5. Standardisasi fitur lalu ulangi klasifikasi.
6. Buat confusion matrix sederhana secara manual.
7. Hitung akurasi pada data uji kecil.
8. Uji beberapa nilai k dan bandingkan hasilnya.
9. Jelaskan apa yang terjadi jika terdapat voting seri.
10. Bandingkan implementasi manual dengan pustaka yang tersedia jika diizinkan.

Bab 12

Praktikum 10: k-Means dan Mini Proyek Data

Tujuan praktikum

Mahasiswa mampu memahami proses iteratif k-means, menentukan stopping rule sederhana, memeriksa perubahan centroid, dan menyusun mini proyek yang dapat dijalankan ulang.

Secara umum k-means mengulang dua langkah: menetapkan setiap titik ke centroid terdekat, lalu memperbarui centroid sebagai rata-rata anggota cluster. Iterasi berhenti ketika assignment atau centroid tidak lagi berubah secara berarti.

Latihan Praktikum 10

1. Lakukan satu iterasi k-means secara manual pada empat titik dan dua centroid awal.
2. Implementasikan assignment ke centroid terdekat.
3. Implementasikan pembaruan centroid.
4. Tambahkan stopping rule berdasarkan perubahan centroid.
5. Catat objective sum of squared distances pada setiap iterasi.
6. Uji beberapa inisialisasi centroid dan bandingkan hasilnya.
7. Visualisasikan cluster untuk data dua dimensi.
8. Jelaskan mengapa skala fitur memengaruhi k-means.
9. Susun folder mini proyek berisi kode, data, output, dan README.
10. Tulis rumusan masalah, input, output, algoritma, validasi, dan cara menjalankan ulang proyek.
11. Buat satu eksperimen tambahan yang mengubah parameter utama dan catat dampaknya.
12. Tulis ringkasan satu halaman yang menjelaskan apa yang dapat dan tidak dapat disimpulkan dari hasil komputasi.

Daftar Pustaka Singkat

1. Miller, B. dan Ranum, D. *Problem Solving with Algorithms and Data Structures Using Python*, Release 3.0.
2. Pine, D. J. *Introduction to Python for Science and Engineering.* CRC Press, 2019.
3. McKinney, W. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*, 3rd ed. O'Reilly, 2022.
4. Dokumentasi Python: `https://docs.python.org/3/`
5. Dokumentasi NumPy: `https://numpy.org/doc/`